

TECHCATALYST DE 2026

Pandas Data Wrangling and Analysis

Make DataFrame transformations explicit, testable, and easy to explain.

Week 3 Pandas

Inspect

Select

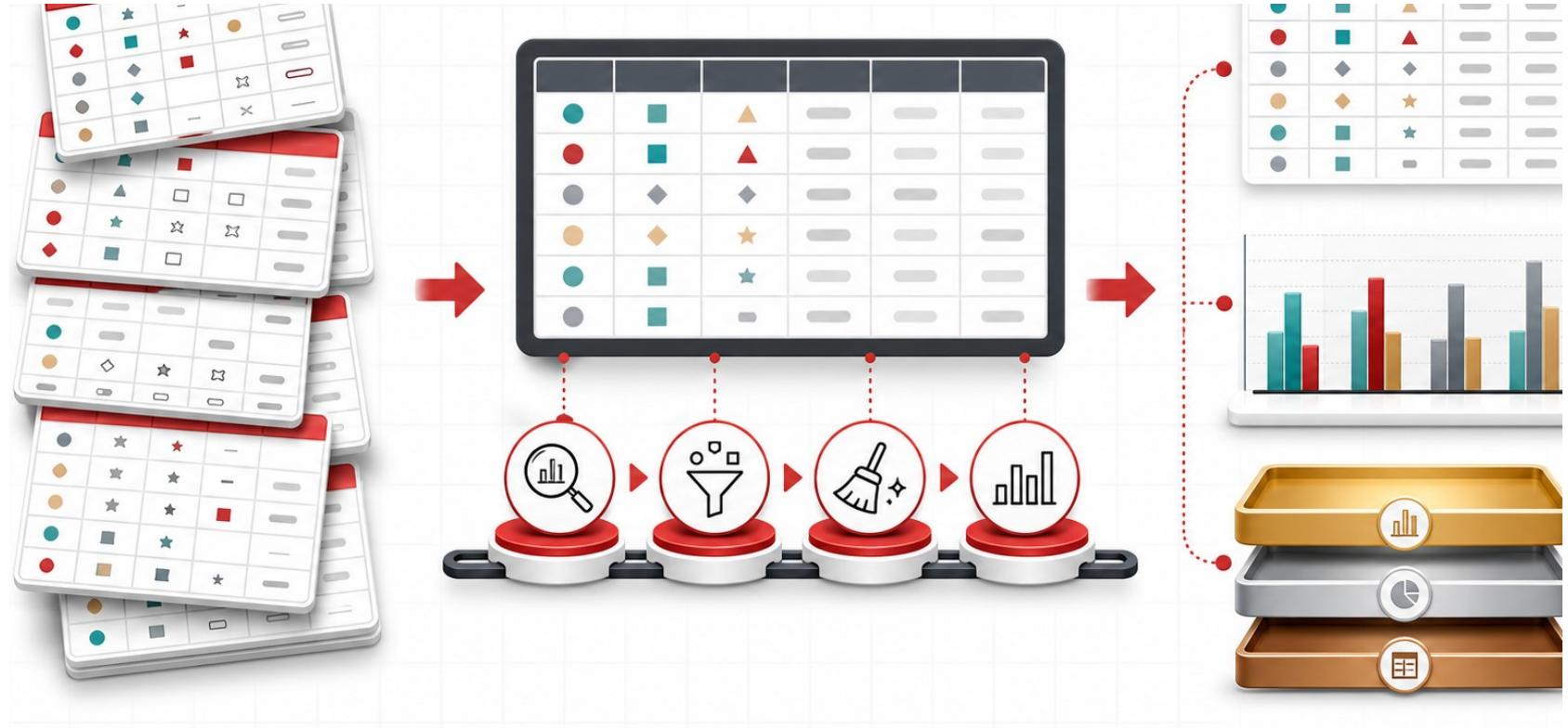
Clean

Group

Merge

Verify

Trust comes from inspection and verification



What you will be able to do

Learning goals	By the end, you can...
Define	State the grain before transforming a DataFrame
Inspect	Check shape, values, types, missingness, and key quality
Select	Choose columns and filter rows with explicit Boolean masks
Clean	Parse types, handle missing values, and remove duplicates
Assign	Use direct assignment and .loc with pandas 3 Copy-on-Write
Summarize	Use groupby and aggregation without losing the grain
Combine	Merge with key expectations and cardinality validation
Verify	Check row counts, types, and persisted Parquet output

Weeks 1 to 4 are an AI-free zone

You cannot review code you could not have written.

What is allowed

- Write and debug your own code
- Use official docs and course references
- Ask peers to explain, not complete
- Call APIs and SDKs when the lesson requires them

Why we do this

- Fluency comes from typing and testing
- Error messages build debugging judgment
- Interviews still test fundamentals
- Later AI use should amplify skills you already own

01

Pandas practice

Inspect first, transform second, verify always.

A DataFrame is a table plus labels and types

Rows are records. Before filtering or joining, decide what one row represents.

Columns are attributes with data types. A numeric-looking object column is still a data-quality problem.

The index labels rows. Use .loc for label-based selection and explicit assignment to the DataFrame.

Inspect before you transform

Check	What it reveals
<code>df.head() / df.sample()</code>	Values, headers, and suspicious formatting
<code>df.shape</code>	Row and column counts
<code>df.info() / df.dtypes</code>	Types and missingness
<code>df.isna().sum()</code>	Missing values by column
<code>df["station"].value_counts()</code>	Category balance and unexpected values

Five checks take two minutes and prevent long debugging sessions.

Select columns and filter rows explicitly

```
cols = ["station", "date", "tmax", "tmin", "prcp"]
weather = df.loc[:, cols].copy()

mask = (weather["tmax"] >= 50) & weather["date"].notna()
warm = weather.loc[mask].copy()

print(warm.shape)
print(warm.head())
```

Use `.loc` for label-based selection. Parenthesize each condition, combine masks with `&` or `|`, and copy when you want an independent working table.

Clean by assigning directly to the DataFrame

```
df["date"] = pd.to_datetime(df["date"], errors="coerce")
df = df.dropna(subset=["date"]).copy()

bad_prcp = df["prcp"].isna() | df["prcp"].lt(0)
df.loc[bad_prcp, "prcp"] = 0

df = df.drop_duplicates(["station", "date"])
df["temp_range"] = df["tmax"] - df["tmin"]
```

pandas 3.0 uses Copy-on-Write by default. Chained assignment no longer updates the parent, so modify the target DataFrame directly with .loc.

Match the operation to the question

Goal	Pandas pattern	Quality question
Summarize	<code>groupby(...).agg(...)</code>	What does one output row represent?
Combine	<code>merge(..., validate=...)</code>	Are the keys unique on each side?
Reshape	<code>pivot_table(...)</code> or <code>melt(...)</code>	Should the result be wide or long?

Always write the grain and expected row-count effect before a group, merge, or reshape.

Close the loop: raw to bronze to silver

Goal: turn 20 messy weather rows into 14 verified silver rows.

Finish the local Pandas path first

- Copy raw bytes unchanged into a dated bronze folder
- Parse dates and remove one failed date
- Dedupe on station + date, then require tmax and tmin
- Repair precipitation and add temp_range plus is_rainy
- Write Parquet, read it back, and assert 14 rows

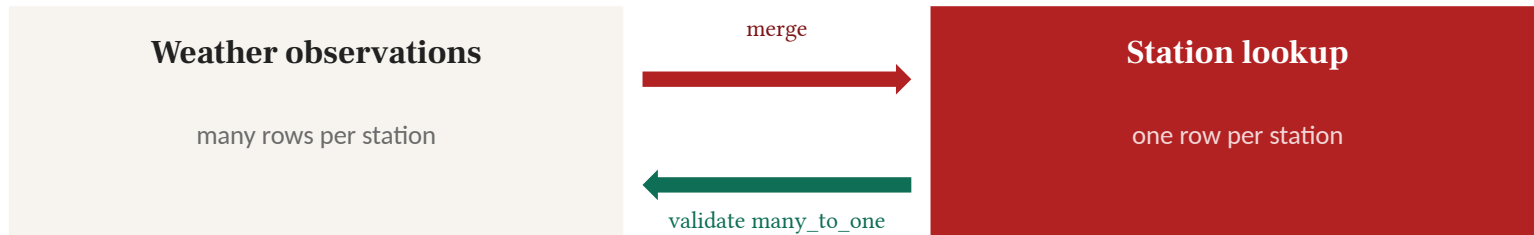
Definition of done: bronze is byte-identical, silver is typed and readable, and your evidence note explains three cleaning decisions.

Group without losing the grain

```
summary = (  
    weather.groupby("station", as_index=False)  
    .agg(  
        days=("date", "nunique"),  
        avg_high=("tmax", "mean"),  
        total_prcp=("prcp", "sum"),  
    )  
)  
  
print(summary)
```

Before grouping, write the output grain: one row per station. Named aggregations make the result easier to inspect and explain.

A merge connects two grains



- State the key on each side
- Check uniqueness before the merge
- Use `validate=` to encode cardinality expectations
- Compare row counts and unmatched keys afterward

Cleaning is a repeatable evidence loop

Inspect: measure types, missingness, duplicates, and suspicious values.

Transform: apply one explicit rule with direct assignment or a focused method.

Verify: recheck the metric and record the row-count effect before the next rule.

DataFrame quality clinic

Goal: explain and repair three quality problems before building a summary.

25 min · pairs

- State the grain and expected key
- Find one type issue, one missing-data issue, and one duplicate
- Apply explicit cleaning rules
- Verify row counts before and after each rule

Stretch: merge a lookup table with `validate=` and explain the chosen cardinality.

Deliverable: a clean DataFrame plus an evidence note for three decisions.

Pandas official references

Reference	Use it for
pandas getting started	DataFrame fundamentals and core operations
pandas indexing guide	.loc, selection, and assignment
pandas Copy-on-Write guide	Current pandas 3 assignment behavior
pandas groupby guide	Split, apply, and combine patterns
pandas merge guide	Joins, keys, and cardinality validation

Checked July 10, 2026. Prefer official documentation when older examples or remembered syntax disagree.

Trust comes from explicit checks

1

Define what one row represents before filtering, grouping, or joining.

2

Inspect shape, values, types, missingness, and duplicates first.

3

Build Boolean masks explicitly and select with `.loc`.

4

In pandas 3, assign directly to the target DataFrame.

5

Verify row counts and keys after every group, merge, and write.

Exit ticket: Which DataFrame check catches your most common mistake?